

Final Project

BMU Verification Plan

List of Abbreviations.....	2
1. Introduction	2
1.1 Purpose	2
1.2 Scope of Verification	2
1.3 Assumptions and Constraints.....	2
2. Design Overview.....	2
3. Verification Strategy	3
3.1 Methodology	3
3.2 Verification Techniques by Category	3
3.3 Features to Verify	3
4. Testbench Structure	4
4.1 Environment Architecture.....	4
4.2 Coverage Targets	4
5. Test Execution	4
5.1 Entry Criteria	5
5.2 Exit Criteria	5
6. Verification Environment Details	5
6.1 Components	5
6.2 Interfaces.....	5
6.3 Tools and Technologies.....	5
6.4 Regression	5
6.5 Coverage & Results Reporting.....	5
7. Risks & Open Items.....	5

List of Abbreviations

Abbreviation	Definition
BMU	Bit Manipulation Unit
UVM	Universal Verification Methodology
DUT	Design Under Test
RTL	Register Transfer Level
RISC-V	Reduced Instruction Set Computer, Version 5
Zbb / Zbs / Zbp / Zba	Basic / Single-bit / Permutation / Address-generation Bit-Manipulation subsets
CSR	Control and Status Register
MSB/LSB	Most/Least Significant Bit
FC	Functional Coverage
TC	Test Case

1. Introduction

1.1 Purpose

This document defines the verification strategy, scope, environment architecture, and success criteria for the Bit Manipulation Unit (BMU). It is the single source of truth for what will be verified and how, and is the parent document for the accompanying Test Plan.

1.2 Scope of Verification

In scope: every BMU operation with a full behavior specification in BMU Specification v1.2 (Section 6), including timing/reset/valid_in behavior (Section 3), guard-condition error detection (Section 4), and the two adopted assumptions (Section 7).

Out of scope, pending clarification: control-struct fields listed in Specification Section 8 (ap.min, ap.clz, ap.rol, ap.bset, ap.bclr, ap.packu, ap.packh, ap.gorc, and the branch-resolution fields). These are tracked as an open risk (Section 7 of this plan) and excluded from coverage goals and exit criteria until the design team confirms scope.

1.3 Assumptions and Constraints

Constraints: none identified at planning time.

- The verification environment uses UVM (class-based, constrained-random capable).
- Functional coverage goals defined in this plan are achievable and measurable within the project timeline.
- Clock and reset are provided correctly by the testbench; not verified as DUT behavior.
- The RTL is synthesizable (not independently re-verified by this project).
- The reference model is built from BMU Specification v1.2, independent of the RTL, so a shared bug cannot hide from the scoreboard.

2. Design Overview

The BMU is a synthesizable, combinational RTL block computing one result per cycle from two 32-bit operands (a_in, b_in) and a decoded control struct (ap), registering the result on the clock edge. It implements the RISC-V Bit Manipulation subset defined in BMU Specification v1.2 — logic, shift, address-generation, arithmetic/comparison, count/population, sign-extension, min/max, pack/permute, and CSR-bypass operations.

2.1 Key Features

- Single-cycle-latency combinational datapath with a registered output (result_ff), enabled by valid_in.
- Configurable instruction enablement via the decoded ap control struct — one primary enable field per operation, with shared-datapath co-requisites for SLT/MAX (adder/subtractor reuse).
- Combinational, always-live error detection, independent of valid_in, covering per-operation guard violations and the three chip-wide conflict conditions.
- Synchronous active-low reset clearing both result and error.

Note: The original design document listed a 'clock-gated PC register' feature. No corresponding PC/branch functionality is documented anywhere in the BMU specification, and its presence could not be confirmed against the documented interface. It has been removed from this plan pending clarification — see Section 7, Risk R-3.

2.2 Interface

Full signal list: BMU Specification v1.2, Section 2. Summary: clk, rst_l, scan_mode, valid_in, a_in[31:0], b_in[31:0], ap (struct), csr_ren_in, csr_rddata_in[31:0] as inputs; result_ff[31:0] and error as outputs.

3. Verification Strategy

3.1 Methodology

UVM (class-based, constrained-random verification) will be used to build a reusable, self-checking environment around the BMU: a driver/monitor/sequencer agent drives and observes the pin-level interface, an independent spec-driven reference model predicts expected result/error for every transaction, a scoreboard compares DUT output against the reference model every cycle, and a functional coverage model tracks which operations, corner cases, and error conditions have been exercised. Both directed sequences (one per documented operation and corner case) and constrained-random sequences (wide operand/field coverage) are used; directed tests establish correctness and traceability per feature, random regression finds unanticipated interactions.

3.2 Verification Techniques by Category

Category	Technique	Rationale
Nominal operation results (per opcode)	Directed	Deterministic expected value; needs precise, traceable coverage per operation.
Corner values (0, all-1s, max shift, sign boundary)	Directed	Random stimulus finds these with low probability; must be forced.
Guard-condition violations / error injection	Directed	Specific field combinations must be constructed deliberately.
valid_in gating / reset timing	Directed	Cycle-accurate timing relationships, not amenable to randomization.
Wide operand space per operation	Constrained-random	Broad confidence beyond hand-picked corner cases.
Multi-operation interaction / unexpected field combinations	Constrained-random	Exercises combinations a directed list would miss.

3.3 Features to Verify

One row per BMU Specification v1.2 section. “Guard errors” denotes the operation's own invalid-condition rows; cross-cutting error conditions are verified once, not per operation.

Spec §	Feature	Key checks
6.1.1	OR / OR-invert	Both modes (ap.zbb); guard errors
6.1.2	XOR / XOR-invert	Both modes (ap.zbb); guard errors
6.2.1	SRL	Shift amounts 0–31; guard errors
6.2.2	SRA	Sign preservation across shift amounts 0–31; guard errors
6.2.3	ROR	Wraparound at boundary amounts; guard errors
6.2.4	BINV	All 32 bit positions; guard errors
6.3.1	SH2ADD	Valid (ap.zba=1); ap.zba=0 invalid case
6.4.1	SUB	Signed wraparound; ap.zba=1 misuse case
6.4.2	SLT / SLTU	Signed and unsigned, including sign-boundary operands
6.5.1	CTZ	a_in=0 → 32 (corrected value); full zero-count range
6.5.2	CPOP	0, 32, and mixed bit-count values
6.6.1	SEXT.B	Positive and negative byte values (bit 7 boundary)
6.7.1	MAX	Both operand orderings; negative vs. positive
6.8.1	PACK	Boundary 16-bit patterns
6.8.2	GREV	Valid b_in=24; adopted-assumption case b_in≠24 (§7.1)
6.9.1	CSR bypass read	ap=0 with csr_ren_in=1 stays valid — the case the AND/OR assumption depends on
6.9.2	CSR write	ap.csr_imm both settings
§3	Latency / valid_in / reset / scan_mode	1-cycle latency; hold on valid_in=0; error live regardless of valid_in; sync reset; scan_mode inert
§5	Chip-wide error conditions	CSR + bit-manip conflict; SUB-in-Zba misuse; SHxADD without ap.zba

4. Testbench Structure

4.1 Environment Architecture

- Agent: driver + monitor + sequencer around the BMU pin interface.
- Reference model: independent predictor built from BMU Specification v1.2 (not derived from RTL).
- Scoreboard: transaction-level comparison of result_ff and error against the reference model, timing-aware (1-cycle latency, valid_in gating).
- Coverage collector: functional coverage groups per Section 4.2, plus code/toggle coverage from the simulator.

4.2 Coverage Targets

Type	Goal
Functional coverage	Cross of {operation} x {corner-case bin} x {guard-valid/guard-violation}, per Section 3.3 feature table. Target 100% of defined bins, or an explicit waiver.
Code (line/statement) coverage	100% of BMU RTL, or waived lines documented with justification.
Toggle coverage	All interface and key internal signal bits toggle 0→1 and 1→0.
Path/branch coverage	All decision points in the operation-select and error-detection logic.
Cross coverage	Operation x valid_in-toggling patterns; operation x reset-during-operation.

5. Test Execution

- One directed test class per feature row in Section 3.3, each running the sequences defined in the Test Plan.
- One constrained-random regression test sweeping legal ap encodings and full operand ranges, run for a fixed high seed count.
- Full regression = all directed tests (fixed seed) + random regression (multiple seeds), executed together via the regression script rather than individually.

5.1 Entry Criteria

- BMU Specification v1.2 reviewed and available; RTL compiles cleanly against the testbench.
- Reference model implemented and unit-checked against the worked examples in the specification.

5.2 Exit Criteria

- 100% of in-scope directed tests passing.
- Functional coverage targets in Section 4.2 met or explicitly waived with justification.
- No open Critical or Major bugs.
- All items in the Spec Clarifications Log resolved, or explicitly accepted as open risk by the design team.

6. Verification Environment Details

6.1 Components

driver, monitor, sequencer, sequence_item, agent, environment, scoreboard, subscriber (coverage).

6.2 Interfaces

A single SystemVerilog interface mediates between the DUT pins and the UVM environment (clocking blocks for drive/sample timing given the 1-cycle latency).

6.3 Tools and Technologies

- Simulator access via MobaXterm.
- Integrated Metrics Center (IMC) for coverage aggregation.
- Cadence Verisium for waveform debugging.
- Visual Studio Code for source editing.

6.4 Regression

All directed and random tests are run together via a single regression entry point rather than individually per test, with pass/fail and coverage summarized per run (see Test Plan status column).

6.5 Coverage & Results Reporting

Coverage reports and final verification results are attached as the project reaches those milestones (see repository docs/07_coverage_reports and docs/08_signoff).

7. Risks & Open Items

ID	Risk	Mitigation
R-1	GREV undefined-encoding behavior (b_in≠24) is an adopted assumption, not confirmed.	Modeled per Specification §7.1; flagged in reference model and test plan; revisit if design team answers differently.
R-2	CSR-conflict AND/OR scoping is an adopted assumption, not confirmed.	Modeled per Specification §7.2, validated against the CSR-bypass-read worked example; revisit if design team answers differently.

ID	Risk	Mitigation
R-3	Control-struct fields with no behavior specification (ap.min, ap.clz, ap.rol, ap.bset, ap.bclr, ap.packu, ap.packh, ap.gorc) and apparently unrelated branch-resolution fields observed in the same module.	Excluded from verification scope and exit criteria until the design team confirms; tracked in Specification §8.
R-4	Timing/reset/scan_mode behavior sourced only from informal clarification, not a reviewed spec revision.	Captured formally in Specification §3; recommend written design-team sign-off before exit.